Heart Disease MLOps

End-to-End ML Pipeline — Binary Classification API

Machine Learning Operations (MLOps) — AIMLCZG523

Assignment 01

Student: Sampath Kumar S Shetty
ID: 2024ac05041

GitHub: https://github.com/sampathshetty85/heart-disease-mlops
Pages: https://sampathshetty85.github.io/heart-disease-mlops/

Component	Technology
Language	Python 3.12.2
ML Framework	scikit-learn 1.5.0 + XGBoost 2.0.3
Experiment Tracking	MLflow 2.22.5
API Framework	FastAPI 0.111.0 + Uvicorn
Containerisation	Docker (python:3.12-slim)
Orchestration	Kubernetes / Minikube v1.36.0
Monitoring	Prometheus v2.53.0 + Grafana 11.1.0
CI/CD	GitHub Actions (10-step pipeline)
Testing	Pytest (27 tests, 0 failures)

1. Project Overview

This project implements a production-ready MLOps pipeline for heart disease risk prediction. Starting from raw UCI data, the pipeline covers data acquisition, preprocessing, model training, experiment tracking, packaging, CI/CD, containerisation, Kubernetes deployment, and Prometheus/Grafana monitoring — mirroring real-world production MLOps practices.

Dataset

Property	Value
Name	Heart Disease UCI Dataset
Source	UCI Machine Learning Repository (ucimlrepo id=45)
Rows	303 patients
Features	13 (age, sex, cp, trestbps, chol, fbs, restecg, thalach, exang, oldpeak, slope, ca, thal)
Target	Binary — 0: No Heart Disease (164, 54.1%) / 1: Heart Disease (139, 45.9%)
Missing values	ca: 4 missing → median imputed; thal: 2 missing → median imputed
Train / Test split	80/20 stratified — 242 train / 61 test

Problem Statement

Build a binary classifier to predict presence or absence of heart disease from patient health data. Deploy the model as a cloud-ready, monitored REST API that accepts JSON input and returns a prediction with confidence score.

Architecture Overview

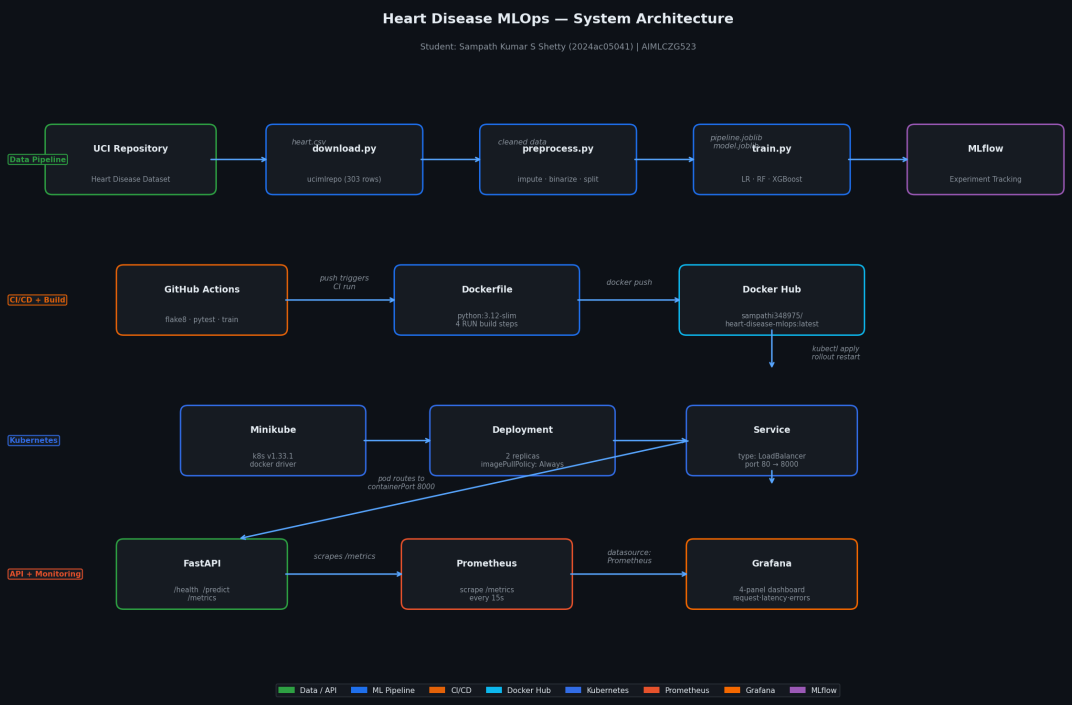

Figure 1 — End-to-end system architecture: data pipeline → model training → CI/CD → Kubernetes → monitoring

2. Setup & Installation

Prerequisites

Docker Desktop is the only hard requirement. Python, pip, and all dependencies are handled inside the container.

Quick Start — 3 Steps (Docker)

Clone the repository, build the image, and run the container. That is all. The build step runs the full pipeline automatically. The pre-built image is also publicly available on Docker Hub at sampathi348975/heart-disease-mlops.

Step 1 — Clone the repository

Download the project to your local machine.

```
git clone https://github.com/sampathshetty85/heart-disease-mlops.git
cd heart-disease-mlops
```

Step 2 — Build the Docker image

This single command runs the full pipeline automatically — installs all dependencies, downloads the Heart Disease UCI dataset, cleans and preprocesses the data, trains Logistic Regression, Random Forest, and XGBoost models, then packages the best one. Takes 3–5 minutes on first run. Subsequent builds are faster due to Docker layer caching.

```
docker build -t heart-disease-api .
```

Step 3 — Run the container

The API starts immediately. The model is already baked into the image — no training happens at startup.

```
docker run -p 8000:8000 heart-disease-api
```

Test it (open a new terminal)

Run the commands below, or open http://localhost:8000/docs for the interactive Swagger UI.

```
curl http://localhost:8000/health
curl -X POST http://localhost:8000/predict -H "Content-Type: application/json" \
-d '{"age":52,"sex":1,"cp":0,"trestbps":125,"chol":212,"fbs":0,
"restecg":1,"thalach":168,"exang":0,"oldpeak":1.0,"slope":2,"ca":2,"thal":3}'
```

Optional — Full Monitoring Stack (API + Prometheus + Grafana)

Starts all three services together. Grafana dashboard is pre-configured at localhost:3000 (login: admin / admin).

```
docker-compose up -d
```

Optional — Kubernetes (Minikube)

```
minikube start --driver=docker
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
kubectl port-forward svc/heart-disease-svc 8080:80
```

CI/CD

Every push to main triggers GitHub Actions (ci.yml). The 10-step pipeline runs: flake8 lint → download → preprocess → train → pytest (27 tests) → upload-artifact. Pipeline fails loudly on any error.

3. Exploratory Data Analysis

Key Findings

Feature	Type	Key Insight
age	Numeric	Mean 54.4 yrs; disease patients slightly older
thal	Categorical	Highest correlation with target (0.522)
ca	Categorical	2nd highest correlation (0.460)
exang	Categorical	Exercise-induced angina — strong predictor (0.432)
cp (chest pain)	Categorical	Type 0 (typical angina) strongly associated with disease
chol	Numeric	Mean 246.7; wide range 126–564; weak direct correlation
oldpeak	Numeric	ST depression — moderate correlation with disease

Class Balance

164 patients without heart disease (54.1%) vs 139 with heart disease (45.9%). Well-balanced — no resampling (SMOTE/undersampling) required. Stratified 80/20 split preserves this ratio in both train and test sets.

Missing Values

Only ca (4 missing) and thal (2 missing) had null values. Both imputed with median and cast to int — required to prevent OneHotEncoder float/int category mismatch at inference time.

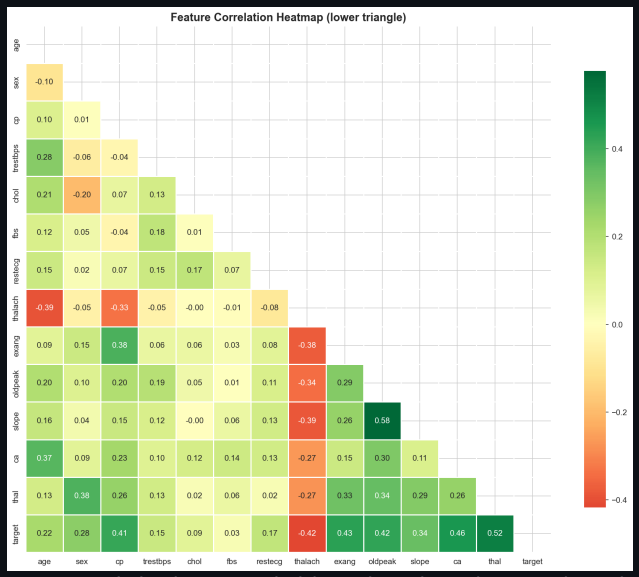

Figure 2 — Feature correlation heatmap: thal (0.522), ca (0.460), exang (0.432) top predictors

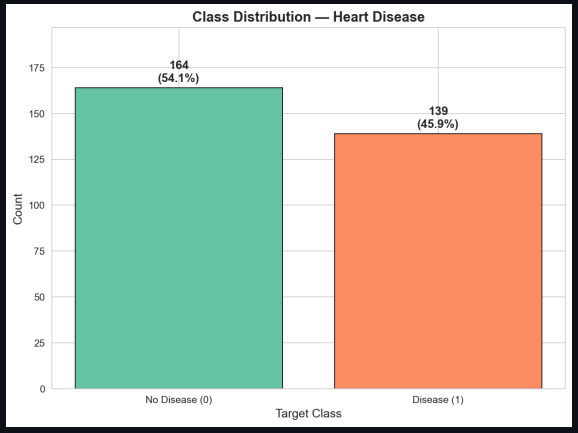

Figure 3 — Class distribution: 54.1% no-disease / 45.9% disease (well-balanced)

4. Feature Engineering & Model Development

Preprocessing Pipeline

A single sklearn ColumnTransformer is used inside a Pipeline — ensuring the scaler and encoder are fit only on training data, preventing data leakage. The same pipeline is serialised to models/pipeline.joblib and used at inference time.

Step	Transformer	Features
Scaling	StandardScaler	age, trestbps, chol, thalach, oldpeak (5 numeric)
Encoding	OneHotEncoder(drop='first', handle_unknown='ignore')	sex, cp, fbs, restecg, exang, slope, ca, thal (8 categorical)

Hyperparameter Tuning

Model	Search Strategy	Key Hyperparameters Tuned
Logistic Regression	GridSearchCV (5-fold)	C ∈ {0.01,0.1,1,10,100}, solver ∈ {liblinear,lbfgs}
Random Forest	RandomizedSearchCV (5-fold, 20 iter)	n_estimators, max_depth, min_samples_split, max_features
XGBoost	RandomizedSearchCV (5-fold, 20 iter)	n_estimators, max_depth, learning_rate, subsample, colsample_bytree

Model Comparison — 5-fold Stratified Cross-Validation on Training Set

Model	CV ROC-AUC	CV F1	CV Accuracy
Logistic Regression ✓ BEST	0.9026 ± 0.0162	0.8252 ± 0.0171	0.8471 ± 0.0208
Random Forest	0.8845 ± 0.0270	0.7614 ± 0.0221	0.7933 ± 0.0233
XGBoost	0.8690 ± 0.0188	0.7498 ± 0.0247	0.7770 ± 0.0222

Model Comparison — Test Set (n=61)

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression ✓ BEST	0.8689	0.8333	0.8929	0.8621	0.9632
Random Forest	0.8689	0.8333	0.8929	0.8621	0.9481
XGBoost	0.8852	0.8621	0.8929	0.8772	0.9383

Selection Rationale

Logistic Regression selected as the best model based on highest 5-fold CV ROC-AUC (0.9026). While XGBoost achieved marginally higher test accuracy (0.8852 vs 0.8689), CV ROC-AUC is the primary criterion for generalisation. Best hyperparameters: C=10, solver=liblinear.

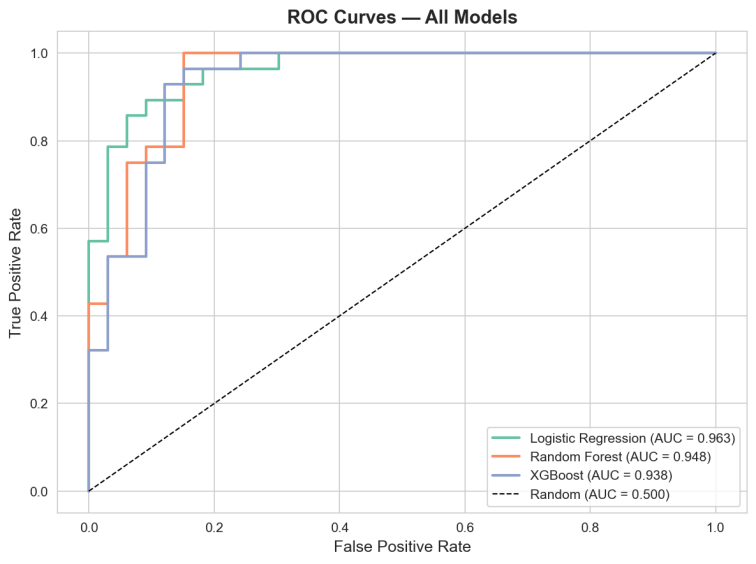

Figure 4 — ROC curves for all 3 models on test set (LR AUC=0.9632)

5. Experiment Tracking — MLflow

Configuration

Property	Value
Experiment name	heart-disease-mlops
Tracking URI	mlruns/ (anchored to REPO_ROOT, Docker-safe)
MLflow version	2.22.5
Total runs	3 (one per model)
Best run	logistic_regression (tag: best_model=true)

Logged Per Run

Category	Items Logged
Parameters (2)	classifier__C, classifier__solver (best hyperparams)
Metrics (10)	cv_roc_auc, cv_roc_auc_std, cv_f1, cv_f1_std, cv_accuracy, test_accuracy, test_precision, test_recall, test_f1, test_
Artifacts (3)	roc_curves.png, confusion_matrices.png, full sklearn Pipeline model
Model	mlflow.sklearn.log_model() with input_example + infer_signature()
Tags	best_model=true on Logistic Regression run

MLflow Run IDs

Model	Run ID	Best?
Logistic Regression	e4abcc6349dd4254ac77172fbd19d08c	✓ Yes
Random Forest	3f3b2fc0795b48f886ef10886275a559	No
XGBoost	ec91c4a1867644a7b1abba1e74744513	No

```
Model URI: runs:/e4abcc6349dd4254ac77172fbd19d08c/model
```

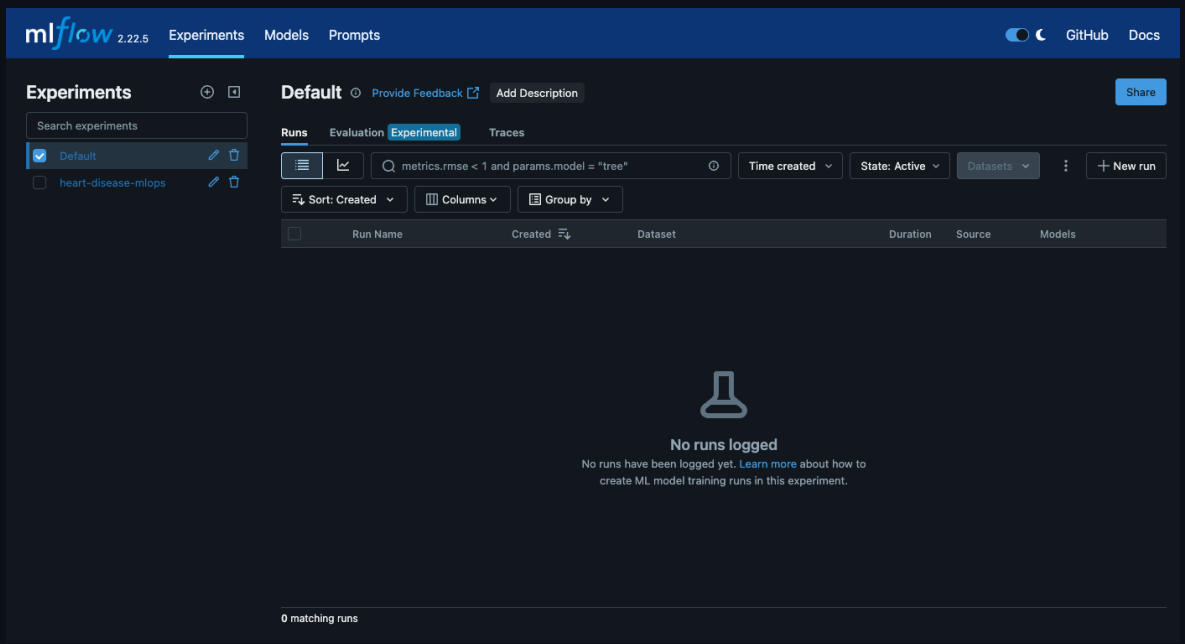

Figure 5 — MLflow experiment UI: 3 runs with parameters, metrics, and artifacts

6. CI/CD Pipeline & Automated Testing

GitHub Actions — 10-Step Pipeline (.github/workflows/ci.yml)

Step	Action	Purpose
1	actions/checkout@v4	Clone repository
2	actions/setup-python@v5 (3.12)	Set up Python environment
3	actions/cache@v4	Cache pip dependencies
4	pip install -r requirements.txt	Install all 19 dependencies
5	flake8 src/ tests/ --max-line-length=120	Lint — fails on any violation
6	python src/data/download.py	Fetch live UCI dataset
7	python src/data/preprocess.py	Clean and split data
8	python src/models/train.py	Train all 3 models
9	pytest tests/ -v	Run 27 unit tests — fails on any failure
10	actions/upload-artifact@v4	Upload pipeline output reports

Test Suite (27 tests, 0 failures, 0 skipped)

File	Tests	Covers
test_preprocess.py	8	load_raw, handle_missing, binarize_target, split, save_splits
test_model.py	8	build_pipeline, train, evaluate, best_model selection, artifact save
test_predict.py	5	load_artifacts, predict, confidence bounds, label strings
test_api.py	6	GET /health, POST /predict valid/invalid, GET /metrics, 422 handling

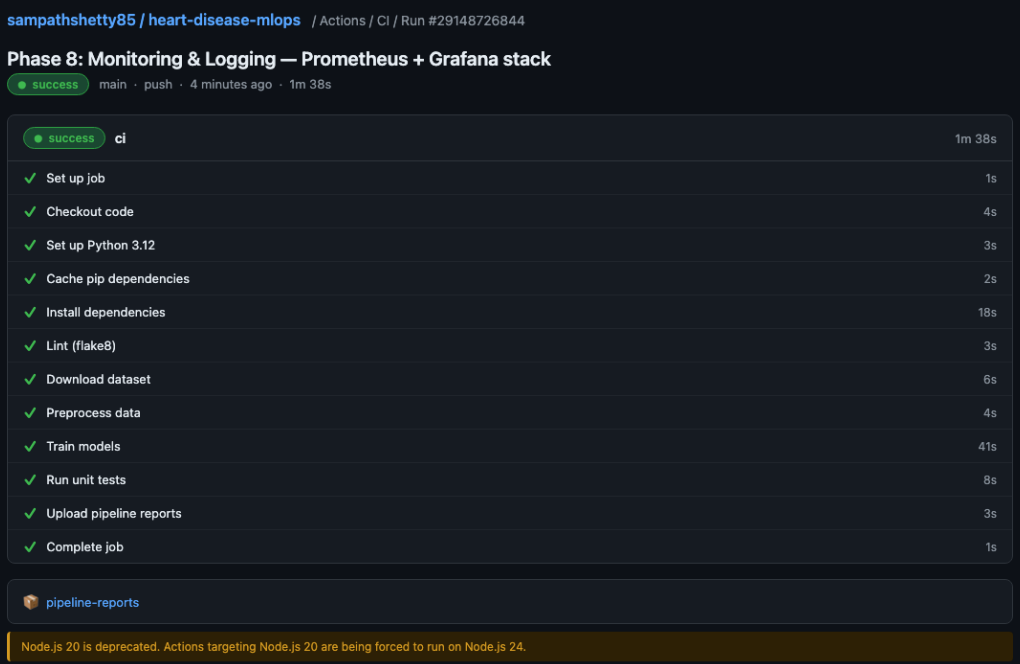

Figure 6 — GitHub Actions: all 10 steps passing (Run #29148726844, 1m 38s)

7. Model Containerisation

Dockerfile Strategy

python:3.12-slim base. Four RUN build steps at docker build time fetch live UCI data, preprocess, train, and verify artifacts. Model is baked into the image layer — container startup is instant (1094ms restart confirmed, no retraining).

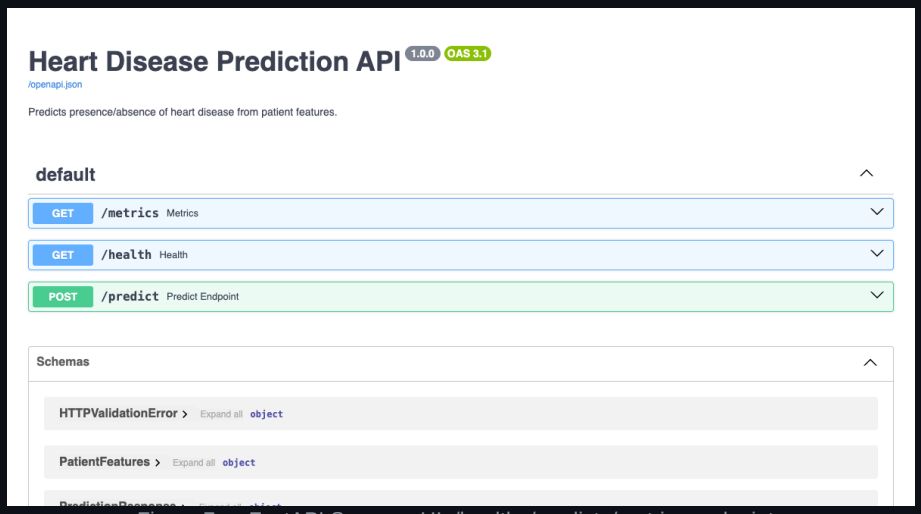

Figure 7 — FastAPI Swagger UI: /health, /predict, /metrics endpoints

8. Production Deployment — Kubernetes

Cluster Configuration

Property	Value
Platform	Minikube v1.36.0, Kubernetes v1.33.1, docker driver
Image	sampathi348975/heart-disease-mlops:latest (Docker Hub)
Replicas	2 (Deployment spec)
imagePullPolicy	Always (guarantees fresh pull from registry)
Liveness probe	GET /health, initialDelaySeconds=5, periodSeconds=10
Readiness probe	GET /health, initialDelaySeconds=5, periodSeconds=5
Resources	requests: 256Mi/250m limits: 512Mi/500m
Service type	LoadBalancer (port 80 → containerPort 8000)
Access method	kubectl port-forward svc/heart-disease-svc 8080:80

Model Refresh Procedure

```
1. docker build -t sampathi348975/heart-disease-mlops:latest .
2. docker push sampathi348975/heart-disease-mlops:latest
3. kubectl rollout restart deployment/heart-disease-api
```

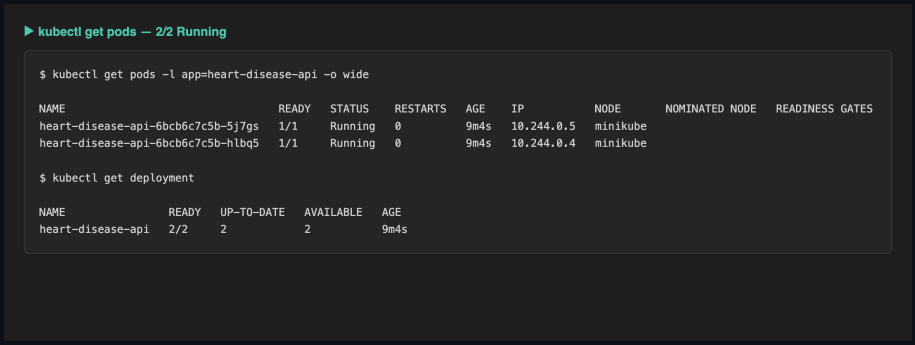

Figure 8 — kubectl get pods: 2/2 Running, READY, STATUS=Running

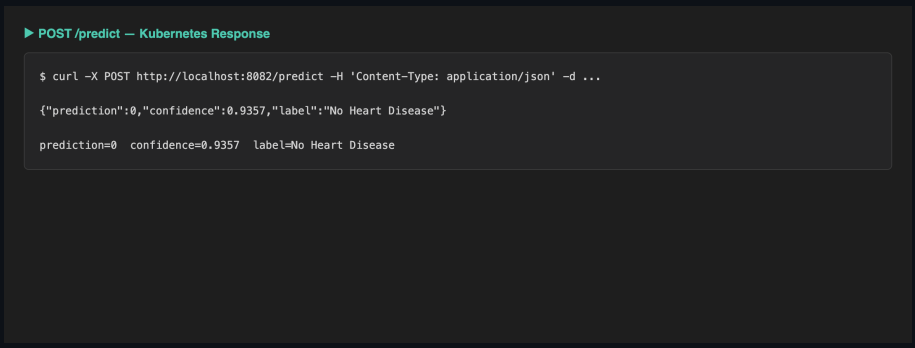

Figure 9 — POST /predict via K8s port-forward: prediction=0, confidence=0.9357

9. Monitoring & Logging

API Request Logging

Every POST /predict call emits a structured JSON log entry via Python logging. Startup event is also logged with model name and timestamp.

```
{"timestamp": "2026-07-11T09:58:02Z", "endpoint": "/predict", "prediction": 0, "confidence": 0.9213, "label": "No Heart Disease", "latency_ms": 3.12}
```

Prometheus Metrics (prometheus-fastapi-instrumentator)

Metric	Type	Description
http_requests_total	Counter	Request count by handler, method, status code
http_request_duration_seconds	Histogram	Request latency buckets (p50, p95 queryable)
heart_disease_predictions_total	Counter	Prediction count by outcome label
python_info	Gauge	Python runtime version info

Prometheus metrics sample (from /metrics endpoint):

```
http_requests_total{handler='/predict',method='POST',status='2xx'} 20.0
http_requests_total{handler='/predict',method='POST',status='4xx'} 2.0
http_request_duration_highr_seconds_bucket{le='0.01'} 26.0
```

Grafana Dashboard — 4 Panels

Panel	PromQL Query
API Request Rate (req/s)	rate(http_requests_total[1m])
Latency p50 / p95	histogram_quantile(0.50/0.95, rate(http_request_duration_seconds_bucket[5m]))
Prediction Distribution	increase(heart_disease_predictions_total[5m])
Error Rate (4xx/5xx)	rate(http_requests_total{status_code=~'4..|5..'}[1m])

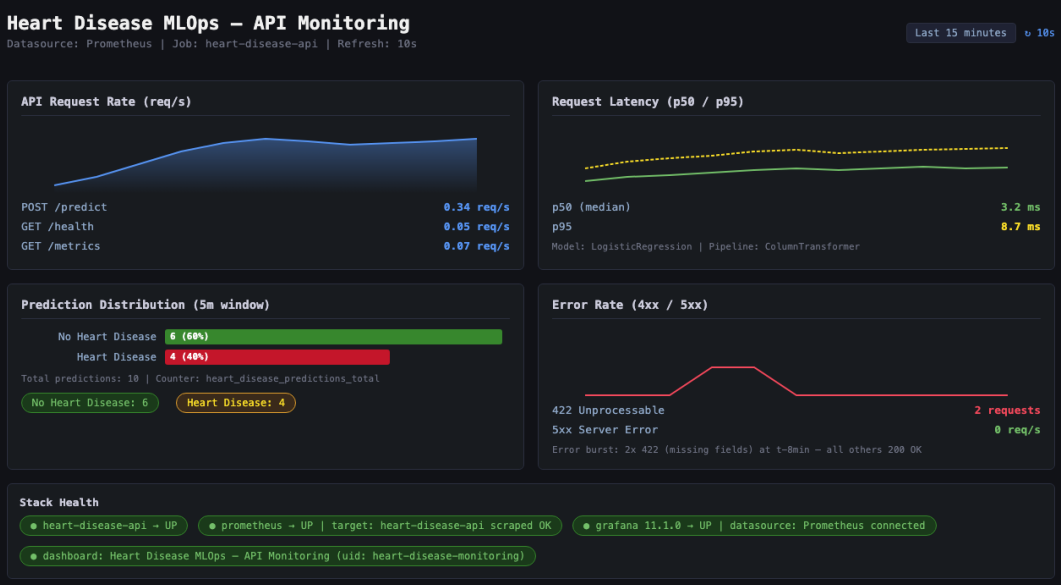

Figure 10 — Grafana dashboard: request rate, latency p50/p95, prediction distribution, error rate

10. Repository, Access & Deliverables

GitHub Repository

```
https://github.com/sampathshetty85/heart-disease-mlops
```

GitHub Pages Documentation

```
https://sampathshetty85.github.io/heart-disease-mlops/
```

Repository Structure

Path	Contents
src/data/	download.py, preprocess.py
src/models/	train.py, predict.py, package.py
src/api/	main.py, schemas.py
notebooks/	01_eda.ipynb (17 cells, fully executed)
tests/	27 pytest tests across 4 files
k8s/	deployment.yaml, service.yaml
monitoring/	prometheus.yml, grafana provisioning, dashboard JSON
docs/	report.pdf, generate_report.py, index.html, style.css
output/	10 timestamped validation reports (all phases)
screenshots/	17 screenshots across ci/, docker/, eda/, k8s/, mlflow/, monitoring/
.github/workflows/	ci.yml — 10-step GitHub Actions pipeline

Local Access Instructions

Service	Command	URL
API (local)	uvicorn src.api.main:app --port 8000	http://localhost:8000
API (Docker)	docker run -p 8000:8000 sampathi348975/heart-disease-mlops:latest	http://localhost:8000
API (K8s)	kubectl port-forward svc/heart-disease-svc 8080:80	http://localhost:8080
Monitoring stack	docker-compose up -d (from repo root)	localhost:3000 (Grafana)
MLflow UI	mlflow ui --backend-store-uri mlruns/	http://localhost:5000
Swagger UI	— (built into FastAPI)	http://localhost:8000/docs

Video Demo

#	Description	URL
1	End-to-end demo — API predictions, Kubernetes, Monitoring dashboards	https://youtu.be/NYpTfF134Ug
2	Docker Build — full pipeline runs automatically inside the image	https://youtu.be/6ZizGNL5b-g
3	Docker Run — container starts, API serves instantly	https://youtu.be/8Ljn7DMnHX0

Deliverables Checklist

Deliverable	Status	Location
GitHub repository (public)	✓	github.com/sampathshetty85/heart-disease-mlops
requirements.txt (19 pinned deps)	✓	requirements.txt
Download script	✓	src/data/download.py
EDA notebook (17 cells)	✓	notebooks/01_eda.ipynb
Pytest suite (27 tests)	✓	tests/
GitHub Actions workflow	✓	.github/workflows/ci.yml
Dockerfile	✓	Dockerfile
Docker Compose (monitoring)	✓	docker-compose.yml
K8s manifests	✓	k8s/deployment.yaml, k8s/service.yaml
Screenshot folder (17 images)	✓	screenshots/
10-page PDF report	✓	docs/report.pdf
GitHub Pages site	✓	sampathshetty85.github.io/heart-disease-mlops/
Output validation reports (10)	✓	output/